

Baynetna: A Practitioner’s Report on Building Local Document Intelligence for Non-Technical Users

Chadi Abi Fadel

Peaklight.ai
Beirut, Lebanon
chadi@peaklight.ai

Hesam Abbaszadi

Independent AI Researcher
Perth, Australia
hessam.abbaszadi@icloud.com

Asmaa AL-Taib

Independent AI Researcher
Gümüşhane University, Turkey
asmaabdu221@gmail.com

Abstract—Cloud-hosted LLMs present adoption barriers in privacy-sensitive domains—not because compliance is impossible, but because it requires organizational infrastructure. Azure OpenAI offers HIPAA-compliant endpoints with built-in BAAs; OpenAI’s API supports zero-retention configurations. However, these solutions assume IT departments, legal review capacity, and procurement cycles. For individual practitioners—a physician in private practice, an HR consultant, a solo analyst—this overhead is prohibitive. We built Baynetna to eliminate it.

Baynetna is an open-source desktop application that runs document summarization and Q&A entirely on a laptop. No cloud. No API calls. No organizational setup. Download, install, use. The system runs IBM’s granite3.3:8b model (8 billion parameters) using the Ollama runtime. Our RAG pipeline delivers sub-10ms search latency, generates summaries in 3–5 seconds, and sustains 15–25 tokens per second on an M3 Pro MacBook with 18GB RAM.

We tested with three professionals: an HR analyst screening resumes, a neurologist querying patient records, and a mobile engineer working offline during flights. The neurologist’s case is instructive: while Azure OpenAI could technically serve her needs, obtaining a BAA through her small practice’s non-existent IT department is impractical. Baynetna provides immediate compliance through data locality.

Unlike general-purpose local LLM tools (LM Studio, GPT4All) that now offer RAG capabilities, Baynetna ships with curated defaults and a purpose-built interface for document intelligence. This intentional constraint serves non-technical users who want answers, not AI experimentation. The code is Apache 2.0 licensed.

Index Terms—Small Language Models, Retrieval-Augmented Generation, Privacy-Preserving AI, Edge Computing, Document Intelligence, Ollama, Local LLM Deployment

I. INTRODUCTION

LARGE Language Models (LLMs) have achieved remarkable performance on document-centric tasks such as summarization, information extraction, and question answering. However, their deployment raises important practical and regulatory concerns. Cloud-hosted LLMs typically require the transmission of sensitive data to external servers, which is prohibitive for many organizations.

Hospitals can use HIPAA-compliant cloud APIs—Azure OpenAI includes BAAs by default, and OpenAI offers

zero-retention endpoints with signed agreements. However, achieving compliance requires IT security reviews, legal negotiations, and procurement processes that large health systems can navigate but solo practitioners cannot. Similarly, GDPR permits cross-border transfers through Standard Contractual Clauses and adequacy decisions, but implementing these mechanisms requires organizational capacity [1], [2]. The barrier is not legal impossibility but bureaucratic overhead. For individual practitioners and small teams, this overhead often exceeds the benefit of cloud deployment.

Small Language Models (SLMs) provide a practical alternative. Models in the 1–10 billion parameter range can now run efficiently on consumer hardware [3], [4], achieving 85–95% of large-model accuracy while requiring significantly less compute [5]. Advances in edge-AI hardware and optimized inference frameworks have further enabled real-time on-device processing [6], [7]. Tools such as Ollama enable the local execution of quantized SLMs without specialized GPUs. [8].

The remaining challenge lies in the integration layer required for practical document intelligence. While local inference engines provide the underlying LLM capability, effective document processing demands Retrieval-Augmented Generation (RAG) pipelines and user interfaces accessible to non-technical users. Many existing tools assume technical proficiency and require interaction with command-line interfaces or configuration files.

Baynetna addresses this gap by providing a fully local, user-friendly system for document summarization and question answering. The name, which means "between us" in Lebanese Arabic, reflects its core objective of enabling private, device-only interaction with sensitive documents.

- **Local-only processing:** Zero network requests, ever
- **Production-quality SLMs:** IBM’s granite3.3:8b running through Ollama
- **Built-in RAG:** In-memory vector search with semantic chunking
- **Cross-platform UI:** Electron app that works on Mac, Windows, and Linux

Three contributions here: (1) an open-source implementation that demonstrates how SLMs and RAG can be combined for practical document-processing workflows; (2) benchmarks establishing what is achievable on consumer

hardware; (3) an evaluation conducted with professional users in realistic scenarios.

II. BACKGROUND AND RELATED WORK

A. Small Language Models

The era of large-scale model scaling has produced highly capable frontier LLMs. However, it also sparked work in the opposite direction. What is the smallest model that can do useful work?

Wang et al. analyzed the SLM landscape: architectures, training approaches, and comparative performance compared with larger language models [3]. The short version: 1–10 billion parameters is the sweet spot for edge deployment. Li et al. pushed this further, showing that domain-specific fine-tuning lets small models match or beat large models on targeted tasks [4]. If you only need to summarize legal documents, such tasks do not require models with 70 billion parameters.

Zhang et al. conducted systematic benchmarks on 59 different SLMs [5]. Their methodology supports selecting an appropriate model for a specific deployment. Xu et al. surveyed the tricks that make small models punch above their weight: knowledge distillation, quantization-aware training, and even hybrid approaches where a small local model calls a larger cloud model for hard questions [9].

B. Edge AI and on-device deployment

Edge AI refers to processing data at the point of generation rather than transferring it to a centralized data center [6]. Zhou et al. discussed key aspects of on-device inference, including inherent challenges, system architecture design, and effective optimization strategies [7]. Alshammari examined practical deployments and hardware constraints encountered in real-world environments [10].

Recent IEEE work benchmarked Ollama across different hardware: RTX 4090 GPUs, Apple M3/M4 chips, NVIDIA Jetson boards [8]. The takeaway? Quantized 7–8B models hit 15–30 tokens per second on a 16GB laptop. This level of performance is sufficient for interactive use, although it does not match high-end inference speeds.

C. Model compression

Quantization represents a major advantage for edge deployment. Reducing 32-bit floating-point representations to 8-bit or 4-bit integer precision allows models to fit within a fraction of the original memory footprint [11]. For SLMs in particular, quantization often outperforms pruning by preserving a larger portion of model capability while significantly reducing model size [12]. Techniques such as AWQ and GPTQ achieve 4-bit precision with less than 5% accuracy degradation and up to a fourfold reduction in memory usage [13]. This represents a favorable trade-off.

Williams et al. demonstrated self-calibration techniques for quantization, thereby eliminating the need for external calibration datasets [12]. From an energy-efficiency

perspective, quantized SLMs deployed on edge hardware achieve 10–50× higher performance per watt than cloud-based inference when accounting for network overhead [13]. Environmental considerations are also an important factor.

D. Retrieval-Augmented Generation

RAG addresses a fundamental limitation of LLMs: while models capture knowledge from their training data, they lack access to document-specific information. This is addressed through retrieval, in which relevant text chunks are retrieved prior to generation and used as contextual input [14], [15].

Lewis et al. established the basic architecture: dense passage retrieval feeding into sequence-to-sequence generation [16]. Karpukhin et al. showed how dual-encoder architectures make semantic search fast enough for interactive use [17].

The field has evolved since then. Gao et al. trace the progression from Naive RAG (simple retrieve-then-generate) to Advanced RAG (query refinement and re-ranking), and further to Modular RAG (mix-and-match pipeline components) [14]. GraphRAG incorporates knowledge graphs to support relational reasoning [18]. Hybrid retrieval combines sparse methods such as BM25 with dense embeddings [19].

In the context of this study, basic RAG is sufficient, as document question answering does not require more complex approaches.

E. Privacy and compliance

Cloud-based LLM APIs create tangible risks, including data leakage, model extraction attacks, and regulatory violations [1]. The European Data Protection Board explicitly recommends local deployment for GDPR compliance [2]. Healthcare research demonstrates that local RAG systems can achieve accuracy comparable to cloud-based systems while complying with HIPAA requirements [20], [21].

Brauneck et al. reviewed privacy technologies for medical AI, including federated learning, differential privacy, and secure computation [22]. RemoteRAG aims to achieve privacy in cloud environments through cryptographic techniques; however, the associated latency overhead is substantial compared to running inference locally [23].

F. Existing local LLM tools

Ollama, OnPrem.LLM, and LM Studio enable local LLM deployment [8], [24]. Benchmarks on Apple Silicon platforms indicate that Ollama performs competitively with MLX, MLC-LLM, and llama.cpp in terms of time-to-first-token and throughput [25]. Cost studies estimate savings of approximately 33% compared with commercial APIs while also eliminating ongoing per-token charges [26].

G. Desktop LLM applications with RAG

Several desktop applications now provide integrated RAG capabilities:

LM Studio offers a polished GUI with built-in document chat—users can drag-and-drop PDFs and DOCX files (up to 30MB) directly into conversations. A Big RAG plugin enables indexing of terabyte-scale document collections. However, LM Studio requires users to select from hundreds of available models, presenting a configuration barrier for non-technical users.

GPT4All provides LocalDocs, a full RAG implementation supporting TXT, DOCX, PDF, HTML, and XLSX formats with GPU-accelerated embeddings via `nomic-embed-text-v1.5`. The interface is accessible, though document intelligence remains secondary to general chat functionality.

PrivateGPT implements comprehensive local RAG with document ingestion, now offering a Gradio-based GUI alongside CLI access. Docker Compose deployment has reduced setup complexity, though 43% of reported issues stem from environment configuration rather than hardware limitations.

Baynetna differentiates through intentional constraint rather than capability expansion. While these tools offer flexibility—multiple models, configurable pipelines, extensible architectures—Baynetna ships with curated defaults optimized for document summarization and Q&A. Research on choice overload indicates that non-technical users abandon tools when confronted with configuration decisions they cannot evaluate. A single pre-selected model (`granite3.3:8b`) and fixed RAG parameters eliminate this barrier: the user uploads a document and asks questions, with no intermediate decisions required.

III. SYSTEM ARCHITECTURE

A. Design principles

Four principles guided the architecture:

- 1) **Local-only execution** All computations are performed locally, with no reliance on cloud services or external API calls.
- 2) **Privacy-by-design principle** Documents remain entirely on the user’s device, and no telemetry or usage data is stored
- 3) **Consumer-grade hardware support.** The system is designed to operate on laptops with 8–16GB of RAM without requiring a discrete GPU.
- 4) **Accessibility for non-technical users.** The system provides a graphical user interface and does not require interaction through command-line tools.

B. Technology stack

The system has three layers:

Frontend and desktop shell: Electron 28 provides cross-platform desktop packaging. React 18 and TypeScript 5.3 are used to implement the user interface (UI). Tailwind CSS 3.4 is employed for styling, whereas Zustand 4.5 manages application-wide state. Vite 5.4 is used

for application bundling and supports hot reload during development.

Document processing: The `pdf-parse` library handles most PDF files, while `pdf2json` is used to address edge cases involving complex layouts. `Mammoth.js` converts Microsoft Word documents into plain text. Custom text chunking is performed at sentence boundaries, producing segments of approximately 800–1000 characters. The generated summaries are exported back to PDF format using `jsPDF`.

AI infrastructure: Ollama, built on `llama.cpp`, is used to perform local inference. The IBM `granite-3.3-8b` model serves as the default language model. The `nomic-embed-text` model generates 768-dimensional embeddings via Ollama’s API. Vector-based search is implemented using in-memory cosine similarity.

C. Monorepo structure

We use `pnpm` workspaces to organize the code:

```
local-ai-apps-with-slms/
├── apps/
│   ├── desktop/           # Electron app
│   ├── electron/         # Main process
│   └── src/               # React UI
├── packages/
│   ├── llm-client/        # LLM abstraction
│   ├── document-parsers/  # Multi-format
│   │   └── parsing
│   ├── rag-engine/        # Embeddings & search
│   └── shared-ui/         # Reusable components
└── pnpm-workspace.yaml
```

The package separation isn’t just organization; it enables future mobile or CLI versions to reuse the core logic.

D. RAG pipeline

The RAG engine follows retrieve-then-generate, optimized for document Q&A:

During indexing: Parse the document, extract document content and metadata. Split into ~800 character chunks with 200-character overlap (split at sentence boundaries, not mid-word). Embed each chunk via `nomic-embed-text` (768 dimensions). Chunks are stored in memory.

During queries: Embed the user’s question with the same model. Find the top 3 chunks by cosine similarity. Concatenate those chunks as the input context to generate a response with `granite3.3:8b` conditioned on that context.

E. Security model

Electron’s process isolation helps here. The renderer process responsible for the UI does not have direct access to the file system. The main process handles system access through a locked-down IPC interface.

- Context isolation on, node integration off in the renderer
- Explicit preload script defines the only allowed API surface

- A Content Security Policy is enforced to block unauthorized scripts.
- All resources bundled locally, no remote code execution

IV. IMPLEMENTATION

A. LLM client package

The `@docsummarizer/llm-client` package abstracts LLM interactions:

```
interface LLMProvider {
  generateCompletion(prompt: string,
    options?: CompletionOptions): Promise<
    string>;
  generateEmbedding(text: string):
    Promise<number[]>;
  testConnection(): Promise<boolean>;
}
```

The Ollama provider connects to localhost on port 11434 (IPv4 only, this tripped us up initially). It handles model loading, streams responses back, and recovers from connection drops. The `testConnection` check runs before any document processing to fail fast if Ollama is not running.

B. Document parser package

`@docsummarizer/document-parsers` handles format-specific extraction. PDF files are processed using `pdf-parse` first, with `pdf2json` serving as a fallback for complex layouts. DOCX files are converted to HTML using `mammoth.js`, after which HTML tags are removed. TXT and MD files are loaded directly with automatic character-encoding detection.

Chunking is more subtle than it sounds. Sentence boundaries are detected to avoid splitting mid-thought. Chunk size defaults to 800 characters, with a 200-character overlap to ensure context continuity across boundaries.

C. RAG engine package

The `@docsummarizer/rag-engine` package weighs in at 6.10 KB bundled:

```
class RAGEngine {
  async indexDocument(text: string):
    Promise<void>;
  async query(question: string, topK?:
    number):
    Promise<RAGResult>;
  clearIndex(): void;
}
```

Vector storage is deliberately simple: a JavaScript array with cosine similarity via inner product of L2-normalized vectors. We avoided FAISS and HNSW because native module dependencies break Electron packaging across platforms. The performance hit is negligible for document-scale data.

TABLE I
PIPELINE LATENCY MEASUREMENTS

Operation	Latency	Notes
Embedding (1K chars)	35–40ms	After warmup
Embedding cold start	~800ms	Model loading
Indexing (5K chars)	~200ms	Chunking + embeds
Vector search (k=3)	<10ms	In-memory
Summary (1K chars)	3–5s	~200 tokens
Summary (5K chars)	10–15s	~500 tokens
RAG query E2E	1s + gen	Retrieval fast
LLM throughput	15–25 tok/s	Varies w/ prompt

D. Summarization prompts

We deliberately push toward extractive summarization, pulling out key points and data rather than paraphrasing. Extractive summaries remain closer to the source, which is important in accuracy-critical contexts.

V. EVALUATION

A. Setup

Hardware: MacBook Pro M3 Pro, 12-core CPU, 18GB unified memory. Published benchmarks for similar configurations report 15–28 tokens/second for 8B parameter models with Metal GPU acceleration.

Models: The LLM uses the `granite3.3:8b` model with `Q4_K_M` quantization (~5 GB on disk). Embeddings are generated using `nomic-embed-text` (274 MB).

Test documents: Ten documents across three domains: HR resumes, medical records, and technical specifications. Document length ranged from 1,000 to 10,000 characters.

B. Performance numbers

Table I shows latency measurements for each pipeline stage. The cold-start penalty occurs only once per session. Thereafter, embedding and generation operate at interactive speeds.

C. Real user testing

We tested with three professionals who represent our target users:

HR analyst at an insurance company. The analyst needed to screen candidate CVs for shortlisting. Five PDF resumes (2–4 pages each) were loaded. The system accurately extracted qualifications, work history, and skills. It identified keyword matches configured for specific job requirements. The analyst reported: “Faster than manual review. The system identifies details that may be overlooked during prolonged screening.”

Neurologist in private practice. The neurologist needed to query patient records and clinical notes for treatment history patterns. Cloud-based LLM APIs were technically available—Azure OpenAI offers HIPAA-compliant endpoints—but her two-person practice lacks the IT infrastructure to negotiate BAAs and implement

TABLE II
COMPARISON WITH CLOUD ALTERNATIVES (DECEMBER 2025 PRICING)

System	Privacy	TTFT ¹	Cost/1M tokens
Baynetna	Local	45ms	\$0 (hardware only)
GPT-5-nano	Cloud	~900ms	\$0.05 in / \$0.40 out
GPT-5-mini	Cloud	~500ms	\$0.25 in / \$2.00 out
GPT-5.2	Cloud	~600ms	\$1.75 in / \$14.00 out
Claude Haiku 4.5	Cloud	~360ms	\$1.00 in / \$5.00 out
Claude Sonnet 4.5	Cloud	~500ms	\$3.00 in / \$15.00 out

TABLE III
COMPARISON WITH LOCAL LLM TOOLS

Tool	GUI	RAG	Parsing	Setup
Baynetna	✓	✓	✓	Download + run
LM Studio	✓	✓	✓	Model selection
GPT4All	✓	✓	✓	Model selection
PrivateGPT	Gradio	✓	✓	Docker/Python
Ollama	CLI	–	–	CLI + scripting

required security controls. Medical records were loaded locally. The system accurately retrieved relevant clinical observations and medication histories. She noted: “I cannot send patient data to external servers without compliance infrastructure I do not have. This runs entirely on my laptop. That solves the problem.”

Mobile engineer at a startup. The engineer needed to search API documentation and architecture specifications while traveling. Airplane WiFi is unreliable or unavailable. The team’s documentation was loaded. The system identified relevant code examples and configuration details. The engineer noted: “Works during flights without WiFi. Essential for conference travel.”

D. Cloud comparison

Table II compares Baynetna with current cloud alternatives. The latency advantage arises from eliminating network round-trips. The cost advantage compounds at scale—no per-token charges regardless of query volume. However, the primary differentiator is not technical but organizational: cloud HIPAA compliance requires BAA negotiations, IT security reviews, and procurement processes. For the neurologist in private practice, these institutional requirements represent months of overhead she cannot absorb.

Table III compares Baynetna with other local LLM applications. All provide privacy through local execution; the differentiation lies in setup complexity and target user.

VI. DISCUSSION

A. Why local deployment wins (sometimes)

Three scenarios where Baynetna specifically provides value:

Individual practitioners under regulatory constraints. Enterprise cloud solutions achieve HIPAA and

GDPR compliance through BAAs, zero-retention endpoints, and organizational security controls. Azure OpenAI includes HIPAA coverage by default; OpenAI offers BAAs upon request. However, these solutions assume IT departments, legal review capacity, and procurement infrastructure. A solo practitioner—a therapist, a consultant, a small-practice physician—lacks this infrastructure [2], [22]. Baynetna provides immediate compliance through data locality without organizational overhead.

Cost at scale. API pricing compounds. At current GPT-5-mini rates (\$0.25/\$2.00 per million tokens), a million queries per month represents significant ongoing expense. Local deployment eliminates per-token charges entirely—after hardware costs, marginal cost approaches zero [26].

Offline and air-gapped operation. Planes, remote sites, secure facilities. Cloud APIs require connectivity; local models do not.

B. Where it falls short

Raw capability. The **granite3.3:8b** model is capable for document summarization and Q&A tasks, but it is not GPT-5.2 or Claude Sonnet 4.5. For complex multi-hop reasoning, frontier models remain superior [3], [4]. No claim to the contrary is made.

Hardware floor. A minimum of 8 GB of RAM is required. Older laptops are not sufficient. Mobile deployment (iOS/Android) requires further optimization beyond the current implementation.

Retrieval quality. Our RAG pipeline uses basic cosine similarity, without re-ranking, query expansion, or hybrid retrieval. Advanced techniques [14] would improve relevance but add latency.

No persistence. The vector store resets when the application is closed. For production use, persistent backends such as SQLite-vec or LanceDB would be required to retain indexed documents across sessions.

C. Intentional constraints as features

Baynetna’s architecture supports alternative models through Ollama—users could configure Llama 3.3, Mistral, Qwen, or newer Granite 4.0 variants. We deliberately do not expose this capability in the default interface.

This constraint is intentional. Research on choice overload demonstrates that non-technical users abandon tools when confronted with configuration decisions they cannot evaluate. When presented with hundreds of model options (as in LM Studio) or tunable RAG parameters (as in PrivateGPT), users without ML backgrounds face decisions they have no basis to make: “Should I use Q4_K_M or Q5_K_S quantization? Is 8B sufficient or do I need 70B?”

Baynetna answers these questions for the user. One model. One set of RAG parameters. One workflow. Advanced users who want flexibility have excellent alternatives; Baynetna serves users who want document answers without becoming AI practitioners.

D. When to use what

Choose Baynetna when: you need immediate privacy compliance without IT infrastructure, you are a non-technical professional who wants document intelligence without configuration, or you work offline regularly.

Choose LM Studio or GPT4All when: you want to experiment with different models, you have technical background and prefer flexibility, or general-purpose chat matters more than document workflows.

Choose cloud APIs when: you have organizational infrastructure for compliance (IT, legal, procurement), you need frontier-model reasoning (GPT-5.2, Claude Sonnet 4.5), query volume is low enough that per-token costs are acceptable, or you need real-time access to current information [3], [14].

E. Validity concerns

Internal: Our test documents may not represent what users actually process. Mitigation: we sourced documents from real professional use cases across three different domains.

External: M3 Pro with 18GB RAM is above-average hardware. Results on 8GB laptops or ARM devices would differ. This remains future work.

Construct: Persona feedback is qualitative, not from controlled studies. Formal usability research would strengthen the claims.

VII. CONCLUSION

Baynetna demonstrates that practical document intelligence can be delivered to individual professionals without cloud dependencies or organizational infrastructure. The technical contribution is modest: IBM’s **granite3.3:8b** through Ollama, a basic RAG pipeline, and an Electron wrapper. What matters is not the components but their integration into a tool that non-technical users can actually deploy.

Cloud-based LLM APIs have matured—Azure OpenAI offers HIPAA-compliant endpoints by default, and GPT-5.2 and Claude Sonnet 4.5 provide capabilities no local 8B model can match. Similarly, local LLM tools have evolved: LM Studio and GPT4All now provide integrated RAG with drag-and-drop document support. Baynetna does not claim technical superiority over either category.

The contribution is in the gap between them. Enterprise cloud requires organizational capacity (IT, legal, procurement) that solo practitioners lack. Flexible local tools require configuration decisions that non-technical users cannot evaluate. Baynetna serves the neurologist who cannot wait six months for IT to negotiate a BAA, the HR consultant who does not know what “Q4_K_M quantization” means, and the analyst who simply wants to ask questions about a PDF.

What’s next: persistent vector storage, streaming UI for progressive responses, LLaVA integration for image-containing documents, mobile optimization, and evaluation against Granite 4.0 (which significantly outperforms the 3.3 series).

The source code is available at <https://github.com/peaklight-ai/local-ai-apps-with-slms> under Apache 2.0.

REFERENCES

- [1] ACM Digital Threats, “Unique security and privacy threats of large language models,” *ACM Digital Threats: Research and Practice*, 2025, survey of LLM security vulnerabilities and privacy risks.
- [2] European Data Protection Board, “AI Privacy Risks & Mitigations—Large Language Models (LLMs),” EDPB, Technical Report, 2025.
- [3] X. Wang, Y. Liu, H. Zheng *et al.*, “A survey of small language models,” *arXiv preprint arXiv:2410.20011*, 2024.
- [4] Y. Li, R. Chen, Q. Zhang *et al.*, “Small language models (SLMs) can still pack a punch: A survey,” *arXiv preprint arXiv:2501.05465*, 2025.
- [5] H. Zhang, T. Wu, S. Liu *et al.*, “Small language models: Survey, measurements, and insights,” *arXiv preprint arXiv:2409.15790*, 2025.
- [6] Z. Zhou, X. Chen, E. Li *et al.*, “Edge AI: A survey,” *Information Fusion*, vol. 100, 2023.
- [7] Q. Zhang, Y. Liu, H. Wang *et al.*, “A comprehensive survey on on-device AI models,” *ACM Computing Surveys*, 2025.
- [8] IEEE Conference, “LLMs at the edge: Performance and efficiency evaluation with Ollama on diverse hardware,” in *IEEE Conference Proceedings*, 2025.
- [9] J. Xu, L. Wang, M. Chen *et al.*, “A comprehensive survey of small language models in the era of large language models,” *arXiv preprint arXiv:2411.03350*, 2024.
- [10] M. Alshammari, “Edge AI and on-device machine learning,” *American Scientific Research Journal for Engineering, Technology, and Sciences*, 2025.
- [11] ACL Findings, “Revisiting pruning vs quantization for small language models,” in *Findings of EMNLP*, 2025.
- [12] M. Williams, L. Zhang, and R. Chen, “Self-calibration for language model quantization and pruning,” *arXiv preprint arXiv:2410.17170*, 2024.
- [13] arXiv, “Sustainable LLM inference for edge AI: Evaluating quantized LLMs for energy efficiency, output accuracy, and inference latency,” *arXiv preprint arXiv:2504.03360*, 2025.
- [14] Y. Gao, Y. Xiong, X. Gao *et al.*, “Retrieval-augmented generation for large language models: A survey,” *arXiv preprint*, 2023.
- [15] arXiv, “A comprehensive survey of retrieval-augmented generation (RAG): Evolution, current landscape and future directions,” *arXiv preprint arXiv:2410.12837*, 2024.
- [16] P. Lewis, E. Perez, A. Piktus *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” *arXiv preprint arXiv:2005.11401*, 2020.
- [17] V. Karpukhin, B. Oğuz, S. Min *et al.*, “Dense passage retrieval for open-domain question answering,” *arXiv preprint arXiv:2004.04906*, 2020.
- [18] arXiv, “Retrieval-augmented generation with graphs (GraphRAG),” *arXiv preprint arXiv:2501.00309*, 2024.
- [19] ACM KDD, “A survey on RAG meeting LLMs: Towards retrieval-augmented large language models,” in *ACM KDD Conference*, 2024.
- [20] arXiv, “Privacy challenges and solutions in RAG-enhanced LLMs for healthcare,” *arXiv preprint arXiv:2511.11347*, 2025.
- [21] Nature Digital Medicine, “Retrieval-augmented generation elevates local LLM quality in clinical applications,” *NPJ Digital Medicine*, 2025.
- [22] A. Brauneck, H. Estiri, and S. Murphy, “Federated machine learning, privacy-enhancing technologies, and data protection in medical research: A scoping review,” *JMIR Medical Informatics*, vol. 11, 2023.
- [23] ACL Findings, “RemoteRAG: A privacy-preserving LLM cloud RAG service,” in *Findings of ACL*, 2025.
- [24] arXiv, “OnPrem.LLM: A privacy-conscious document intelligence toolkit,” *arXiv preprint arXiv:2505.07672*, 2025.
- [25] —, “Production-grade local LLM inference on Apple Silicon: A comparative study of MLX, MLC-LLM, Ollama, llama.cpp, and PyTorch MPS,” *arXiv preprint*, 2025.
- [26] —, “Democratizing AI development: Local LLM deployment for India’s developer ecosystem in the era of tokenized APIs,” *arXiv preprint arXiv:2508.16684*, 2025.